PhD Literature Reviews

Bruno Costa Alves Freire

September 25, 2025

1 Introduction

This document is meant to assemble drafts of literature review notes.

1.1 Target literature

1.1.1 PhD topic specific reading list reading

- (3.2) [1] Modern Coding Theory
- (3.1) [2] Efficient Erasure Correcting Codes
- (2.1) [3] Quantum LDPC codes with positive rate and minimum distance proportional to $n^{1/2}$
- (2.2) [4] Decoding Across the Quantum LDPC Code Landscape
- () [5] Fast erasure decoder for hypergraph product codes
- (2.3) [6] Asymptotically Good Quantum and Locally Testable Classical LDPC Codes
- () [7] Quantum Tanner codes
- () [8] Quantum Expander Codes
- () [9] Combining hard and soft decoders for hypergraph product codes
- () [10] Efficient soft-output decoders for the surface code
- () [11] Weight Reduced Stabilizer Codes with Lower Overhead
- () [12] High-threshold, low-overhead and single-shot decodable fault-tolerant quantum memory
- () [13] Ambiguity Clustering: an accurate and efficient decoder for qLDPC codes
- () [14] On the iterative decoding of sparse quantum codes
- () [15] A topological theory for qLDPC: non-Clifford gates and magic state fountain on homological product codes with constant rate and beyond the $N^{1/3}$ distance barrier
- () [16] Asymptotically good CSS-T codes exist
- () [17] Computing Efficiently in QLDPC Codes
- () [18] A recursive approach to low complexity codes

1.1.2 General FTQC literacy

- () [19] Scheme for reducing decoherence in quantum computer memory
- () [20] Error Correcting Codes in Quantum Theory
- () [21] Stabilizer Codes and Quantum Error Correction
- () [22] Topological quantum memory
- () [23] Surface codes: Towards practical large-scale quantum computation
- () [24] Sparse Blossom: correcting a million errors per core second with minimum-weight matching
- () [25] Targeted Clifford logical gates for hypergraph product codes

1.1.3 Notes from NISQ2LSQ workshop in Marseille 22-24/01/2025

- Nicolas SAUSSAY : A variant of the Bravyi-Terhal bound for arbitrary boundary conditions. (3.3.1 for notes);
- Wouter ROZENDAAL : An upper bound on the minimum distance of Abelian two-block group algebra codes. (3.3.2 for notes);
- Thomas VAN HIMBEECK : Can we achieve the hashing bound with small weight quantum codes? (3.3.3 for notes);
- Clément POIRSON : Engineering CSS surgery to perform logical free phase ZX-diagrams. (3.3.4 for notes);
- Paul HILAIRE : Tailoring QEC codes to a photonic FTQC architecture. (3.3.5 for notes);
- Mike VASMER : Single-shot and measurement-based quantum error correction via fault complexes. (3.3.6 for notes);
- Samuel NOVAK : Basics of rotors: what is it, and why is it $\mathbb{Z}$? (3.3.7 for notes);
- Christophe VUILLOT : Tiger codes. (3.3.8 for notes);
- Gilles ZEMOR : Protecting Clifford circuits with rank-metric techniques. (3.3.9 for notes);
- Virgile GUEMARD : Moderate length quantum Tanner codes. (3.3.10 for notes);

1.1.4 On the decoding vs. decoder benchmarking for quantum erasure channels question

When actually decoding an erasure, in the classical case, one needs to output a codeword (or the completion of it) in function of the non-erased part of the received message. In the quantum case, the output should be a correction to be applied on the qubits, that must be compatible with a measured syndrome, and restricted to the support of the erasure. In both cases, if the decoder is not allowed to give up (signal failure), it is responsible for outputting something, that is, it should either be sure that there is a unique answer, or guess one among the equally likely possibilities. The point here is, for an actual decoder, this output depends (even if in a simple way) on something more than just the erased subset (the non-erased bits in the classical case, the measured syndromes in the quantum case). However, if we are interested solely in measuring the failure probability of the decoder (assuming it fails whenever it's not sure of the output), i.e., benchmarking it, we don't really need any information apart from the erasure subset.

This fact is simple enough in the classical case (cf. Richardson & Urbanke section 3.6.1). In the quantum case, let us provide a clearer argument as to why this is also true. Let $H_X, H_Z \in \mathbb{F}_2^{R\times N}$ define a CSS code $\mathcal{Q}$ (for simplicity, the same would work for any stabilizer code), let $\mathcal{E} \subset [N]$ be the erasure subset, and $E \leq \mathcal{E}$ is the error pattern resulting from the syndrome measurement (that is, if the initial state was $|\psi\rangle \in \mathcal{Q}$, the state after the syndrome measurement is $E|\psi\rangle$). For simplicity, we'll refer to the X/Z component of E with

the same notation, meaning $E \in \mathbb{F}_2^N$ and $\operatorname{supp} E \subset \mathcal{E}$. Let $\sigma_{Z/X} = H_{Z/X} E$ denote the syndrome measured at the Z/X stabilizers (capturing the X/Z component of E, resp.).

Define $\mathcal{C}(\mathcal{E}, \sigma) := \{E \in \mathbb{F}_2^N\big|_{\mathcal{E}} : H_Z\big|_{\mathcal{E}} E = \sigma\}$ as the set of corrections that are compatible with the syndrome σ and supported inside the erasure $\mathcal{E}$. Notice that $\mathcal{C}(\mathcal{E}, \sigma)$ is a vector subspace of $\mathbb{F}_2^N\big|_{\mathcal{E}}$. Moreover, let $\mathcal{S}_X = \operatorname{im} H_X^T = \ker \eta(H_X^T) \subset \mathbb{F}_2^N$ denote the group of X-type stabilizers, also seen as a subspace of $\mathbb{F}_2^N$, and its restriction $\mathcal{S}_X\big|_{\mathcal{E}} := \{S\big|_{\mathcal{E}} : S \in \mathcal{S}_X, \operatorname{supp} S \subset \mathcal{E}\} = \ker \eta(H_X^T)\big|_{\mathcal{E}}$ formed by the stabilizers that are supported in the erasure.

The correctability condition of the erasure $\mathcal{E}$ is given by

$$\mathcal{E} \text{ is correctable} \iff \frac{\mathcal{C}(\mathcal{E}, \sigma)}{\mathcal{S}_X\big|_{\mathcal{E}}} \cong \{\overline{0}\},$$

that is, when we quotient the subspace of compatible corrections by the stabilizer group, we should have only one equivalence class, otherwise, there will be logical errors hiding in the erasure. What we claim here is that, for all possible syndromes σ, we'll have $\#\frac{\mathcal{C}(\mathcal{E}, \sigma)}{\mathcal{S}_X\big|_{\mathcal{E}}} = \#\frac{\mathcal{C}(\mathcal{E}, 0)}{\mathcal{S}_X\big|_{\mathcal{E}}}$, which will in turn imply that the correctability condition is independent of the particular syndrome.

To prove this, we'll first assume that the syndrome σ is caused by an error supported in the erasure, i.e., $\exists E \in \mathbb{F}_2^N$ s.t. $H_Z E = \sigma$ and $\operatorname{supp} E \subset \mathcal{E}$. That implies that $\mathcal{C}(\mathcal{E}, \sigma) \neq \varnothing$. Thus, let $\overline{C} \in \frac{\mathcal{C}(\mathcal{E}, \sigma)}{\mathcal{S}_X\big|_{\mathcal{E}}}$, meaning $\overline{C} = C + \mathcal{S}_X\big|_{\mathcal{E}}$, $C \leq \mathcal{E}$ and $H_Z\big|_{\mathcal{E}} C = \sigma$. It is then easy to see that $\frac{\mathcal{C}(\mathcal{E}, \sigma)}{\mathcal{S}_X\big|_{\mathcal{E}}} = \frac{\mathcal{C}(\mathcal{E}, 0)}{\mathcal{S}_X\big|_{\mathcal{E}}} + \overline{C}$.

That proves that the correctability condition is independent on the syndrome, which means that the performance of the maximum likelihood decoder can be assessed with random erasure samples assuming only trivial syndrome. The same can be said for other decoders such as peeling and its variants, to the extend that their progression is not impacted by the particular value of the syndrome. In the following, let us analyze in more detail the operation of peeling, pruned peeling and branching to further convince ourselves of this fact.

Analyze the progression of peeling

1.1.5 Conjectures about the capabilities of branching vs. peeling and ML

1. At the asymptotic regime ($p \to 0$), the branching algorithm can match ML's performance if the number of available guesses g is at least $d - s$, where d is the code distance and s is the stopping distance.

Handwavy argument: For an erasure rate of p, the logical error rate is given by $\sum_{\mathcal{E} \subset [N]} p^{|\mathcal{E}|}(1-p)^{N-|\mathcal{E}|} \cdot [\mathcal{E} \text{ leads to a decoding failure}]$. Consider $\mathcal{E}$ such that $|\mathcal{E}| < d$. That implies that, for this instance of erasure error, the ML algorithm would succeed, i.e., $\mathcal{E}$ cannot contain a nontrivial logical operator. Under this assumption, the branching algorithm should only fail if it runs out of guesses before peeling $\mathcal{E}$ completely. Now, we know that for any erasure $\mathcal{E}'$ with $|\mathcal{E}'| < s$ (the stopping distance), the peeling algorithm is able to completely peel off $\mathcal{E}'$ and thus succeed. Since we assume that branching has at least $g = d - s$ guesses to spare, and it starts with an erasure pattern $\mathcal{E}$ of size $\mathcal{E} < d$, in the worst-case scenario peeling will block and trigger the branching mechanism as long as we have guesses left. However, after $d - s$ guesses, the size of the residual erasure will be less than $d - (d - s) = s$, and so it cannot still be a stopping set. That is, after at most $d - s$ guesses, the residual erasure gets too small to stop peeling from proceeding until the end. Thus, we get to the end with some number of branches (which may or may not have been cut along the execution), which we know must be all valid (that is, syndrome compliant and contained in the original erasure) and stabilizer-equivalent. To see why the remaining branches must be valid, just recall that branching has a consistency-check mechanism every time if finds a parity-check equation that had become dangling but whose dangling bit has been unerased before it was evaluated. Whenever these so-called empty equations are evaluated, they catch any branches that cause an inconsistency for it and trigger their cutting. Finally, to see why all the branches must be stabilizer-equivalent, just recall that, since $|\mathcal{E}| < d$, it cannot contain a nontrivial logical operator, and thus, all syndrome and erasure-compliant corrections must be stabilizer-equivalent. This way, we have that, for all $\mathcal{E}$ such that $|\mathcal{E}| < d$, $[\mathcal{E} \text{ leads to a decoding failure}] = 0$, and thus

$$p_L = \sum_{|\mathcal{E}|\geq d} p^{|\mathcal{E}|}(1-p)^{N-|\mathcal{E}|} \cdot [\mathcal{E} \text{ leads to a decoding failure}] = O(p^d).$$

Now, notice that the above argument gives us the same decay exponent d as for the ML algorithm, but we can't say the same about the constant, that is, how many erasure patterns $\mathcal{E}$ of size d will lead to a decoding failure for branching? Certainly at least as many as for ML, but could there be erasure patterns of size d for which ML succeeds and branching still fails even with $g = d - s$ guesses? Possibly yes. Nothing rules out the existence of d-sized erasure patterns that don't cover logical operators, but that contain s-sized stopping sets. To get rid of those, it suffices to add just one more guess, so $g \geq d - s + 1$ would be enough to match ML's asymptotic performance in both exponent and constant factor. It's good to highlight that it does not mean that the logical error rate for both algorithms will be exactly the same, because we can still have very large erasure patterns for which ML succeeds, but that block peeling sufficiently many times to exhaust all guesses lead to branching failure. However, those large patterns will have a negligible contribution to the logical error rate as $p \to 0$, in comparison to the others.

1.1.6 On the difference between *in silico* simulations vs. experimental determinations of logical error rates

So far, whenever I did simulations of the performance of codes, be it for the erasure channel or Pauli channels, for whatever decoder, the overall pipeline was simple: first, I sample some error pattern (Pauli string or erasure subset), compute the syndromes from it (I always assumed perfect measurements, but I could also include measurement error without changing much the pipeline), decode the syndrome/erasure (if solely for benchmarking purposes, I could just check whether the decoder is successful or not, not necessarily computing its output) and finally checking whether the predicted error/correction successfully brings the system back to the correct codestate. In all this procedure, not only are the simulations independent of the initial codestate (they don't affect any aspect of the decoding and assessment of correctness of the decoder's output), but they are also independent of data qubit measurements. The problem is, by keeping track of the physical error that affects the data (and eventually also the ancilla) qubits, I was doing something that is not compatible with an experimental setup. In experiments, we can only have access to information that we measure, and thus the whole benchmarking pipeline I described is impossible.

How then can we measure the performance of a code/decoder? How can we tell whether the decoder's correction brings us back to the initial codestate? In the *in silico* simulations, the criterion was basically that the *residual error* (that is, physical error minus decoder's correction) should be a stabilizer. That means we should make sure to return to the codespace (which is guaranteed in the absence of measurement error), and furthermore return to the same codestate as before. Well, the residual error is not accessible in the experimental setup. Moreover, since the measurements are noisy, and assuming that the decoder operates only with input from ancilla measurements, we don't have a guarantee of returning to the codespace. For the purpose of benchmarking the code/decoder in a memory experiment, we can allow ourselves to measure the data qubits at the end (but not feed them to the decoder), so we need to somehow extract (measure) information about this residual error from the data qubits.

It is easy to see that, if the residual error is not a stabilizer, it may flip some of the stabilizers (in which case it takes us out of the codespace), or, if it doesn't, then it is a (non trivial) logical operator, in which case it will flip some other logical operators, e.g., the Z_L logical operator is flipped by X_L or Y_L, assuming each of those act on the same logical qubit. If we assume that our logical qubit register is analogous to a physical register, and that each logical qubit undergoes, independently (or not?) some error channel (let's consider only Pauli channels at first), we could model the logical error channel as a Kraus map

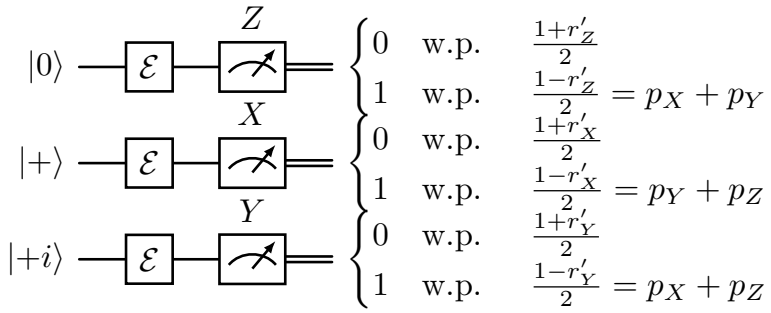
$$\mathcal{E}_L : \rho \mapsto (1-p_L)\rho + \sum_{W\in\{X,Y,Z\}} p_{W,L} W_L \rho W_L,$$

and set out experiments to estimate the parameters $p_{W,L}$. Note, however, that the W_L operators are only well defined if we assume that $\rho = |\psi\rangle\langle\psi|$ is in the codespace, because they are in fact *cosets* of operators by the stabilizer group. Should we include some probability of getting out of the codespace? In any case, how do we go about estimating those parameters?

In the physical setting, what we do is quantum process tomography. If we had a physical Kraus map of the form $\mathcal{E}(\rho) = (1-p)\rho + p_X X\rho X + p_Y Y\rho Y + p_Z Z\rho Z$ (where $p = p_X + p_Y + p_Z$), we could estimate its parameters by looking at its action on a density operator $\rho = \frac{1}{2}(I + \vec{r}\cdot\vec{\sigma})$,

$$\mathcal{E}(\rho) = \frac{I + [(1-p) + p_X - p_Y - p_Z]r_X X + [\dots]}{2} = \frac{I + \sum_{W\in\{X,Y,Z\}}[1-2(p-p_W)]r_W W}{2}.$$

Define $s_W = 1 - 2(p - p_W)$ as the *shrinking factor* in the W axis, $W \in \{X, Y, Z\}$. The resulting density operator $\rho' = \mathcal{E}(\rho)$ is given by $\frac{1}{2}(I + \vec{r'}\cdot\vec{\sigma})$, where $\vec{r'}$ describes the position of the physical qubits in the Bloch sphere after undergoing the error channel $\mathcal{E}$. By preparing ρ in different initial states and measuring ρ' in different bases, we can estimate the shrinking factors s_W, and thus the Pauli error parameters p_W, with the following circuits:



By preparing the initial states so that we have $r_W = 1$, we measure 1 (or -1 in the $+1/-1$ convention) in the corresponding basis with probability $\frac{1-s_W}{2}$, which will give us a linear system that can be further solved for p_W.

We can think of doing a similar thing for logical qubits. Initialize the register in the logical $|0\rangle_L$, $|+\rangle_L$ or $|+i\rangle_L$ states, run some cycles of syndrome extraction followed by the application of the decoder's correction, and then measure the logical Z_L, X_L or Y_L operators. There are some problems here though. The logical operators have multiple representatives, and the measurements of those will not disagree if we are not in the codespace.

Questions:

- When benchmarking codes/decoders experimentally, should we feed the decoder with data qubit measurements?
- Assuming the decoder can't use data qubit measurements to compute the correction, we have no guarantee of returning to the codespace. Thus, different representatives of logical operators will disagree upon measurement. What should we measure then?
- For general stabilizer codes, how do we go about measuring a logical observable from the data qubits? How many of such observables can we measure simultaneously (i.e., in the same shot)? Certainly at most k (= # of logical qubits) since there are pairs of logical operators that anti-commute.
- How do we compatibilize the definition of logical error rate from the *in silico* computations with the QPT approach described here? Keeping in mind that in the former definition, any nontrivial logical operator yields a logical error, but depending on the initial codestate, some nontrivial logical operators leave the state unchanged (e.g., if we are in the logical $|0\rangle_L$ state, the logical Z_L operator doesn't amount to a logical error). That is, what the *in silico* simulations compute is akin to $p_L = p_{X,L} + p_{Y,L} + p_{Z,L}$. What should we use as a figure of merit then?

2 Short analyses and comments

2.1 HGP codes

Key paper by Tillich and Zémor, introduced the HGP code construction and changed the state-of-the-art of QEC codes in terms of the asymptotics of the minimum distance parameter (previous SOTA was $d = O(\log n)$, upgraded to $d = \Theta(\sqrt{n})$), for qLDPC codes with fixed rate (i.e., $k = \Theta(n)$). The paper presents the HGP construction and draws connections with related concepts, such as hypergraphs, explains how the HGP codes generalize toric codes, and proves the asymptotic properties of HGP codes. The main advantage of this construction is it's capability of creating CSS codes from any two classical codes, while preserving the LDPC property.

2.1.1 Further reflections and remarks

During the internship we overlooked a problem related to the code with parameters $[\![1225, 65]\!]$. The classical code used to produce this HGP has $m = 21$ equations and $n = 28$ bits, but it's parity-check matrix has rank only 20, so the transpose code is nontrivial, which is why we can even get an HGP code whose dimension is not a perfect square to begin with. The problem is, as we deform the code to traverse the state space, the rank of the parity-check matrix may increase, leading to the reduction of the resulting HGP code's dimension. The lowest dimension we can get is for when the classical parity-check matrix is full-rank, which is the case for the other three families, hence why we overlooked this problem during our investigations and didn't bother checking for a decrease of the codes' dimension.

This sheds light on the fact that the HGP construction *is not a function of the two classical codes*, but rather of their *parity-check matrix / Tanner graph representations*. The same pair of classical codes can have different matrix and graphical representations, which yield effectively different HGP codes. Adding or suppressing redundant parity-check equations (rows in the matrix) doesn't change the underlying code, but adds or suppresses check nodes in the Tanner graph, all of which will directly reflect on number of physical qubits of the HGP code. Moreover, replacing equations by linear combinations of the others implies changing the connectivity structure of the Tanner graph, going as far as changing the number of edges and the degree distributions, so the LDPC property is also perturbed by this kind of change. From another point of view, the HGP construction depends on the transpose codes (or at least their matrices and graphs), so applying row equivalent transformations to H translates into column equivalent transformations to H^T, which don't necessarily preserve the transpose code. So far, the only identified transformations that can be applied to matrices or graphs while preserving in some sense the HGP code are row/column permutations or graph isomorphisms. I wonder if the same could be said about any isometric isomorphisms (i.e., any invertible linear transformation that preserves the Hamming distance) of the classical code.

In sync with the discussion on 3.2, the choice of the specific matrix or graph representation of the code may be irrelevant for ML decoding, but it becomes relevant for iterative decoding algorithms (possibly also for encoding?). For the HGP construction, regardless of the decoding approach, the code itself is different according to the matrix/graph representation choice of the classical codes.

2.2 Decoding qLDPC codes

This paper proposes a decoding strategy for HGP codes based on a combination of Belief Propagation (BP) and Order Statistics decoding (OSD). There is a github repo with the code available.

2.3 Lifted product codes

Important paper by Panteleev and Kalachev, first to introduce asymptotically good quantum LDPC codes. Asymptotically good means that the codes have both linear dimension and minimum distance, $[\![n, k = \Theta(n), d = \Theta(n)]\!]$. The construction is based on an operation from homological algebra known as lifting, hence the name lifted product codes.

2.4

3 In-depth reading

3.1 Efficient erasure correcting codes (classical)

3.1.1 Questions to keep in mind

- Question: Can we tailor HGP codes for which the simple peeling decoder works well? It is known that for HGP of two regular codes, peeling alone doesn't work, but how about irregular codes?
- More precisely: given some erasure rate p, what is the maximum rate k/n of a HGP code such that the peeling decoder succeeds with high probability?

3.1.2 Notes

- *Linear-time encoding/decoding for transmission just below the capacity.* Given $\epsilon > 0$, with rate $R = 1 - (1+\epsilon)p$ (capacity is $1-p$), encode and decode with time complexity $O(n \ln(1/\epsilon))$.
- The code construction is related to LDPC, but the secret ingredient is irregularity. Carefully chosen degree sequences (performance is a function of this degree sequence).
- Analysis tools: differential equations to study the progress of the decoding algorithm, combinatorial arguments to prove the success of the decoding, linear programming to find good degree sequences.
- Definition of graph codes: Let B be a bipartite graph with k left nodes and βk right nodes ($0 < \beta < 1$), denoting respectively the message bits and the check (or redundant) bits. Encode the message by setting each check bit as the XOR of its message neighbors (done in time proportional to the number of edges). Not the same as a Tanner graph!
- Algorithm 1 (erasure decoding): For a given check bit, if only one of its message bits is erased, set it as the XOR of the check bit and all its non-erased message neighbors. This idea is similar to the concept of a dangling check as per [5]. Is there really any difference between these graph codes and just Tanner graphs? It seems equivalent, just as Algorithm 1 seems equivalent to the peeling decoder from [5].
- Here comes the difference: to handle erasures of both message and check bits, they introduce the cascading of bipartite graphs. Basically, treat the check bits as messages bits of a new graph code, producing a new layer of bits. Iterate the process until the final layer of bits is small enough to be corrected efficiently with a conventional code.
- The decoding of these cascade graphs works by correcting the erasures of each layer from the rightmost to the leftmost, so that at each iteration, the check bits are corrected and can be used to decode the message bits. In this sense, it seems we recover the same description as the peeling decoder.
- The graphs chosen are very important: random sparse bipartite graphs, with a carefully chosen degree sequence (irregular).
- Specify graphs via the edge degrees: $(\lambda_i)_{i=1}^m$, λ_i for the initial fraction of edges with degree i on the left, $(\rho_j)_{j=1}^m$, ρ_j for the initial fraction of edges with degree j on the right. Degree d on the left/right meaning that the left/right endpoint of that edge is a node with degree d. Introduce generating polynomials $\lambda(x) = \sum_i \lambda_i x^{i-1}$, $\rho(x) = \sum_j \rho_j x^{j-1}$.
- Typo in the formula of $\Lambda(x)$, the correct should be $a_\ell \cdot \int_0^x \lambda(t)\mathrm{d}t = (\sum_i x^i \lambda_i/i)/(\sum_i \lambda_i/i)$.
- Existence of graphs with given degree sequences rely on an inductive argument (verify). Moreover, the probability distribution of the random subgraphs obtained along the decoding process is claimed to be uniform over subgraphs with given degree sequences (verify).

3.2 Modern Coding Theory

3.2.1 Quick remarks about Chapter 1

The goal is not to read the whole book but I found it fruitful to read even superficially the chapters preceding chapter 3 to get a more grounded view of the subject (and to spare the need to keep going back to learn definitions/conventions established in the previous chapters).

The point of *modern* coding theory, in opposition to the classical theory (classical not as in non-quantum, but rather as traditional or foundational), seems to be the attention given to the complexities of the encoding and decoding procedures. Classical coding theory is all about finding block codes with a large minimum distance and an algebraic structure that allows for the design of efficient decoders. Efficient in terms of error probability, not necessarily of low algorithmic complexity. The modern theory on the other hand is concerned with these algorithmic complexities, and is not so much relying on codes that have a particular algebraic structure (at most we ask for linear codes), but rather on codes that can perform well under simpler decoders (iterative decoders mostly). The graphical representations become very important in this sense, and message-passing decoders receive a lot of attention. In terms of efficiency (low error probability), we may even aim lower than the capacity of the channel, for the sake of having fast encoders/decoders at some rate that can still deliver low error probability.

3.2.2 Quick remarks about Chapter 2

- In chapter 2 they introduce the notion of message-passing algorithms. First in the context of computing marginals with factor graphs, but later it gets contextualized to bitwise MAP decoding.
- Messages are essentially functions of a finite alphabet, i.e., the same as vectors. For the binary alphabet, they become 2D real-valued (because they encode probabilities) vectors. The fact of being 2D allow us to talk instead in terms of the ratio $r = \mu(0)/\mu(1)$, or the log-likelihood ratio $l = \ln(r)$.
- In terms of the log-likelihood, the message passing rules can be written as sums (at the variable nodes, $l = \sum_k l_k$) and multiplication of hyperbolic tangents (at the check nodes, $\tanh(l/2) = \prod_j \tanh(l_j/2)$).
- To minimize bit error probability, we work on the sum-product semiring, whereas for block decoding, we work on the max-sum semiring. Initial messages are likelihoods for bit decoding, whereas for block decoding we use log-likelihoods. For discrete output channels, it is convenient to replace max-sum by min-sum instead.
- In general, message-passing works if the factor graphs are trees. However, codes whose Tanner graphs are trees will inevitably have many low-weight codewords and hence a high probability of error.
- For Tanner graphs with cycles, message-passing can also work but won't be MAP, i.e., it will be strictly suboptimal. In chapter 3 we shall study the performance of that combination.

3.2.3 Chapter 3 notes

- Understand the notion of degradation. For $\delta < \epsilon$, a BEC(δ) can be written as the concatenation of the receiving end of a BEC(ϵ) with a channel that adds some more erasure. Author claims this notion will be useful eventually.
- Remark: notice that in the BEC, the MAP (block) decoder outputs something only if there's no ambiguity about the sent codeword, i.e., it either succeeds or gives up, no chance of error. The same will hold for the peeling decoder. For the MAP decoder to succeed, $H_{\mathcal{E}}$ must be full-rank.
- Remark on Tanner graphs: just as the parity-check matrix of a code is not unique, the same goes for the graph (and nope, they are not all isomorphic either, because you can combine rows of H which may add/subtract edges, in general not simply permuting the nodes). Different Tanner graphs for the same code may have different performances under message-passing decoders.

- Remark on code ensembles: $\mathcal{H}(n,k)$ and $\mathcal{G}(n,k)$ are basically random matrices $\mathcal{B}(1/2)^{\otimes(n-k)\times n}$ and $\mathcal{B}(1/2)^{\otimes k\times n}$, and are not the same (actual rate may differ from design rate in different directions). For message-passing analysis, we care about a Tanner graph ensemble, which is easy to define in terms of degree sequences, but has some side effects such as allowing for multigraphs and not taking graph isomorphism into account. As a result, the distribution over the induced codes is not quite uniform.

- Convention: left nodes are bit nodes, check nodes are right nodes.

- Define the polynomials $\Lambda(x) = \sum_i \Lambda_i x^i$ where Λ_i is the number of degree i left nodes. Analogously for $P(x)$ and right nodes. Normalized versions $L(x) = \Lambda(x)/\Lambda(1)$, $R(x) = P(x)/P(1)$. LDPC(Λ, P) ensemble is given by bipartite graphs with $\Lambda(1), P(1)$ left/right nodes; Λ_i, P_i left/right nodes have i sockets for edges, the connection between left and right sockets is a permutation of the total number of edges chosen uniformly at random.

- Degree sequences from the edge perspective: $\lambda(x) = \Lambda'(x)/\Lambda'(1)$, analogously for $\rho(x)$ with $P(x)$.

- Degree one variable nodes are bad, don't tolerate them. They can be introduced if you are careful, see Chapter 7.

- Actual rates vs. design rates, under some conditions the two are equal with high probability, and this concentration is stronger for large n. For regular ensembles the conditions are satisfied, for example.

- Moving on to message-passing decoders on Tanner graphs containing cycles, we need to set schedules. Since we only care about the ratio of the likelihoods, we can simplify the messages from $(1-\epsilon, 0)$, (ϵ, ϵ), $(0, 1-\epsilon)$ to $(1,0)$, $(1,1)$, $(0,1)$. These messages will be denoted respectively as the zero, the erasure and the one messages. Initial messages for the BEC(ϵ), $(p_{Y_j|X_j}(y_j|0), p_{Y_j|X_j}(y_j|1))$:

y_j \ x_j	0	1
0	$1-\epsilon$	0
?	ϵ	ϵ
1	0	$1-\epsilon$

  $\xrightarrow{\text{normalization}}$

y_j \ x_j	0	1
0	1	0
?	1	1
1	0	1

- Message passing rules at variable nodes: agree on the common non-? incoming message, if any, else, ?. In other words, the outgoing message is a ? only if *all* the incoming messages are ?, otherwise all the non-? are the same because the channel doesn't introduce errors. In the 2D tuple representations, this amounts to doing a component-wise AND.

- Message passing rules at check nodes: basically XOR all the incoming messages, with the rule that $? \oplus 0 = ? \oplus 1 = ?$. I.e., the outgoing message is a ? if *any* of the incoming messages is a ?. Otherwise, just XOR them all (in the $0, ?, 1$ representation).

- To clarify the message passing thing, a node will send over one of its edges, the outgoing message computed from the incoming messages *that come from the remaining edges only*, i.e., not considering the message that came from the edge through which it is sending now. E.g., if node n has edges e_1, e_2, e_3, the message sent out through e_2 will be a function of the messages that came in from edges e_1, e_3. Also, at the variable nodes, we consider that y is an incoming message at every iteration.

- Simplification no. 1: everything is independent of the sent codeword x, so we may assume it is the all-zeros codeword wlog.

- Simplification no. 2: due to Theorem 3.30 that states that the failure rate of BP for a given Tanner graph G gets concentrated around the ensemble average as $\mathbb{P}[|P_L(G) - \mathbb{E}_{G'\sim\text{LDPC}}[P_L(G')]| > \delta] \leq e^{\alpha n}$, the authors will focus on designing ensembles with good average performance. Notice, however, that this concentration kicks in as $n \to \infty$, so maybe it isn't as good for moderate sized codes.

- Refer to appendix C for techniques and tools related to Theorem 3.30 (and its proof). When explaining the gist of this theorem, the authors claim that the cross-wiring action that we know and love is meant to have little impact (because local) on the performance of a code under BP decoding. They then claim

that any two LDPC codes can be converted into each other via a linear number of such actions and each modification has a negligible impact on the performance, so all the codes in the ensemble should behave similarly. Again, in the limit of large blocklength.

- In the following section, we discuss the performance of BP for LDPC codes assuming the two simplifications. For such, we introduce notions such as the computation graph and its ensemble. The gist of the computation graph is to model the neighborhood of a variable node in the Tanner graph, and by controlling its depth, we can compute the probability of failure for a given number of BP iterations. The goal then will be to consider ensemble averages of such failure rate, and then take the limit as $n \to \infty$.

- Next, we introduce the notion of tree ensembles. The idea is that as the blocklength grows to infinity, the computation graphs will become trees, with high probability. So, since we're interested in this asymptotic regime, let us study the computation graph ensembles assuming that they are trees. These ensembles can be defined from the degree distributions λ, ρ. To deal with the fact that a computation graph for a fixed code may not be a tree, we consider the notions of a tree code and the projection of a code onto a computation graph. Then lemma 3.47 states that a computation graph is proper with high probability.

- All this computation graph, tree ensemble and whatnot amounts to the convergence to the tree channel theorem, which basically states that

$$\lim_{n\to\infty} \mathbb{E}_{\mathrm{LDPC}(n,\lambda,\rho)}[P_b^{\mathrm{BP}}(G,\epsilon,\ell)] = P^{\mathrm{BP}}_{\mathcal{T}_\ell(\lambda,\rho)}(\epsilon).$$

  Notice that the number of BP iterations is kept fixed here. Basically, when $n \to \infty$, the average performance of a code from the LDPC ensemble under BP (with ℓ iterations) decoding converges to the performance of the corresponding tree channel. And what is the performance of the tree channel?

- Density evolution: defining $x_{-1} = 1$, we have $x_\ell = \epsilon\lambda(1-\rho(1-x_{\ell-1}))$, and the performance of the tree channel is given by the expressions $P^{\mathrm{BP}}_{\mathcal{T}_\ell}(\epsilon) = x_\ell$ and $P^{\mathrm{BP}}_{\mathcal{T}_\ell}(\epsilon) = \epsilon L(1-\rho(1-x_{\ell-1}))$. The proof is done by induction, and it relies on the careful construction of the tree ensembles to work seamlessly. Now, notice that from this perspective, we are now considering how the performance of BP evolves as we increase the number of iterations ℓ.

- The density evolution theorem is followed by results about the monotonicity of the evolution. Basically, the performance of BP is monotonic wrt to the channel parameter ϵ and the sequence x_ℓ converges to the closest fixed point of the equation $x = \epsilon\lambda(1-\rho(1-x))$. With these monotonicity results, one can define the threshold of a given degree distribution pair (λ, ρ) as

$$\epsilon^{\mathrm{BP}}(\lambda,\rho) = \sup\{0 \le \epsilon \le 1 : P^{\mathrm{BP}}_{\mathcal{T}_\ell(\lambda,\rho)}(\epsilon) \xrightarrow{\ell\to\infty} 0\}$$

- Continue from section 3.12: Fixed point characterization of threshold. Actually, skip to at least 3.19. Then fast-forward to 3.22. Keep pushing until 3.24, that's where the things we are interested in right now start appearing.

$$\Lambda(x) = \sum_i \Lambda_i x^i \quad P(x) = \sum_i P_i x^i$$
$$L(x) = \Lambda(x)/\Lambda(1) \quad R(x) = P(x)/P(1)$$
$$\lambda(x) = \Lambda'(x)/\Lambda'(1) \quad \rho(x) = P'(x)/P'(1)$$
$$(\Lambda, \Gamma)$$

$$P^{\mathrm{BP}}_{\mathcal{T}_t}(\epsilon) = \epsilon L(1-\rho(1-x_{t-1})) \quad P^{\mathrm{BP}}_{\mathcal{T}_t}(\epsilon) = x_t$$

$$x_t = \epsilon\lambda(1 - \rho(1 - x_{t-1})), \quad x_{-1} := 1$$

$$\sum_i \rho_i[1 - (1 - x_t)^{i-1}] = 1 - \rho(1 - x_t)$$

3.3 Notes from the NISQ2LSQ Marseille workshop

I tried to take notes of all talks, some rather unsuccessfully. Instead, I highlight the parts I understood and which I think are relevant or interesting.

3.3.1 A variant of the Bravyi-Terhal bound for arbitrary boundary conditions

This work is focused on probing the theoretical limits of geometrically-local codes, more precisely giving upper bounds on their minimum distance depending on the geometric dimension of their embedding. Basically, in geometrically-local codes, your qubits are mapped to points in $\mathbb{Z}^D/\Lambda$, where Λ is some lattice. The most important result on that is the Bravyi-Terhal bound, which basically states that $d_{\min} \leq rL^{D-1} = O(N^{\frac{D-1}{D}})$. So, in particular, for 2D local codes, one has $d = O(\sqrt{N})$.

The proof of such bound (and of the generalized bound from this work) relies on the Cleaning Lemma. Bravyi-Terhal applies for lattices of the form $(\mathbb{Z}/L\mathbb{Z})^D$. The generalization here refers to taking more general lattices. In summary, yes, the bound still applies, with somewhat worse constants, but asymptotically it's the same. Among the theoretical tools used in their proof, they introduce a mapping for the qubits onto the lattice, and the concept of volume of a lattice. The new proof still relies mainly on the cleaning lemma.

Other interesting points were the mention of GB (Generalized Bicycle) codes, which are CSS codes formed by a pair of circulant matrices associated to polynomials in $\mathbb{F}_2[x]/(x^N-1)$, e.g.,

$$A = \begin{pmatrix} | & | & \cdots & | \\ a & x\cdot a & \cdots & x^{N-1}\cdot a \\ | & | & \cdots & | \end{pmatrix} \quad \text{where} \quad [a] = \begin{pmatrix} a_0 \\ \vdots \\ a_{N-1} \end{pmatrix} \quad \text{for} \quad a \in \mathbb{F}_2[x]/(x^N-1),$$

so the CSS code is defined as $H_X = \big(A|B\big)$, $H_Z = \big(B^T|A^T\big)$. For reference about GB codes, see ECZoo entry. Also, check this out: Matching Generalized-Bicycle Codes to Neutral Atoms for Low-Overhead Fault-Tolerance!!!! Looks super relevant for Pasqal.

3.3.2 An upper bound on the minimum distance of Abelian two-block group algebra codes

I can't seem to recall the result they presented, but I'm pretty sure it's just the same Bravyi-Terhal bound but for A2BGA codes. If anything new, they relate it to the weight of the stabilizers instead of the dimension of the embedding. Most interesting part was the whole background on the algebraic constructions.

- Group algebra: take a finite field $\mathbb{F}$ and an abelian group G of size $|G| = \ell$, and define the group algebra $\mathbb{F}[G] := \{\sum_{g\in G} x_g g : x_g \in \mathbb{F}\}$. Basically formal sums.
- They have a product formula $(\sum_g x_g g)(\sum_g y_g g) = \sum_k (\sum_{gh=k} x_g y_h)k$. With this, $\mathbb{F}[G]$ becomes an $\mathbb{F}$-algebra.
- Define the permutation matrix $\mathbb{B}(g) = [\mathbb{B}(g)]_{i,j} = 1 \iff g_i g_j^{-1} = g$. With these you can define the Two-Block (2B) codes (I missed this slide). See ECZoo entry.
- Abelian 2BGA codes: they have parameters $[\![n = 2\ell, k = n - \operatorname{rank} H_X - \operatorname{rank} H_Z, d = d_X = d_Z]\!]$. CSS matrices follow the same construction as in GB, but here $A = \sum_g a_g \mathbb{B}(g), B = \sum_g b_g \mathbb{B}(g) \in \mathcal{M}_l(\mathbb{F}_2)$.
- 2BGA codes generalize GB codes by taking $G = C_\ell$ (the cyclic group of length ℓ). Enter Bivariate-Bicycle codes: take $G = C_r \times C_s$.
- $a, b \in \mathbb{F}_2[C_r \times C_s] \cong \mathbb{F}_2[x,y]/\langle x^r - 1, y^s - 1, xy - yx\rangle$. Then define $X = P_r \otimes I_s, Y = I_r \otimes P_s$, then $A = a(X,Y), B = b(X,Y) \in \mathcal{M}_{rs}(\mathbb{F}_2)$. Example: the Gross code $[\![144, 12, 12]\!]$.
- cf ECZoo entry.

3.3.3 Can we achieve the hashing bound with small weight quantum codes?

Thomas is a postdoc with Anthony now at COSMIQ. His work seems very relevant and in connection to what I'm doing now. LST paper. Highlights were the LST constructions, graph states, weight distributions and their expected value computed via techniques from the book Modern Coding Theory.

3.3.4 Engineering CSS surgery to perform logical free phase ZX-diagrams

In summary it is about generalizing lattice surgery from surface codes to CSS codes. There are some notions from ZX calculus that triggered Jean-Pierre a bit. The formalism is heavy loaded with homological algebra, and the notions of subcodes and merges are important.

3.3.5 Tailoring QEC codes to a photonic FTQC architecture

Absorbed very little of this one. Basically Paul Hilaire is from Quandela, and he showed some results about their implementation of Floquet codes (i.e. Dynamical QEC) and that it was better than plain surface codes.

3.3.6 Single-shot and measurement-based quantum error correction via fault complexes

Talk by Mike Vasmer, new permanent at Inria, coming from Canada (Xanadu). The talk relies again on homological algebra, characterizing CSS codes via chain complexes, but with extra elements. He discusses the single-shot property, which can be interestingly framed as: instead of requiring $O(d)$ measurement rounds for good correction performance, require only $O(1)$. Again mentions graph states (related to MBQC). Define a new construction called *foliation*, which is a recipe to construct a FT graph state from a CSS code. Finally, define fault complexes, built as the tensor product of two chain complexes.

3.3.7 Basics of rotors: what is it, and why is it Z?

Very mathy, super cool. Rotors lie in-between discrete and continuous variable notions of quantum codes. The basis is to take the circle $\mathbb{S}^1 = \mathbb{R}/2\pi\mathbb{Z} = \mathbb{T}$ as the space for the position operator $\psi \in \mathcal{L}^2(\mathbb{T})$, which in turn leads to the momentum operator being described by $\mathbb{Z}$, or $\hat{\psi} \in \ell^2(\mathbb{Z})$. Next, you define translation operators on both spaces ($\mathbb{T}$ and $\mathbb{Z}$), and define X and Z as complex exponentials of such translation operators. Compute their commutation relations, generalize them to N rotors. Next, define maps from $\mathbb{Z}^N$ and $\mathbb{T}^N$ to the Pauli group $\mathcal{P}_N$. Apply Fourier Transforms on a LCA (locally-compact, abelian) group, the dual of $\mathcal{L}^2(G)$ is $\mathcal{L}^2(\hat{G})$ where $\hat{G}$ is the Pontryaguin dual of G, or the characters of G. Next, the same old recipe for building CSS codes via chain complexes, but now it is over $\mathbb{Z}$-modules. Towards the end, you have the construction of some codes via tesselation of manifolds, in particular the projective plane $\mathbb{RP}^2$.

3.3.8 Tiger codes

Super awesome. Tiger codes generalize cat codes, they're like cat codes on steroids. Recall quantum modes and bosonic codes, recall Fock states and coherent states, the number and the annihilation/creation operators. Define number and annihilation constraints. The gist of tiger codes comprises syndromes and dissipators. Define projected coherent states. The normalization depends on this obscure function called the Gelfand-Kapranov-Zelevinsky hypergeometric function $A_\Delta(y)$. Lots and lots of maths and physics and too many details that I couldn't manage to absorb completely. Refer to the paper Letting the tiger out of its cage: bosonic coding without concatenation. Overall, it kind of builds on top of the rotor codes talk and also some previously introduced concepts.

3.3.9 Protecting Clifford circuits with rank-metric techniques

Gilles's talk was hard to grasp. He urged us to learn the fundamentals of network coding. Work is very much on the line of fixing an amount t of errors you aim to correct. Seems like we have to condensate ℓ qubits into one, effectively working with 2^ℓ-qudits. Define the rank metric. Mentioned Gabidulin codes and his recent collab with Delfosse.

3.3.10 Moderate length quantum Tanner codes

Virgile presents many interesting tools. First there is the concept of lifts, i.e. graph covering. Then he defines the Sipser-Spielman construction of Tanner codes (still in the classical setting). There is the concept of a Lifted Tanner code. And finally he translates all that into the quantum case. The idea of his work is to lie somewhere between codes that are very implementable but scale like shit, e.g. toric codes, and codes that

are hard to even conceive of but have the GOAT scalability properties, like the asymptotically good codes, PK21, LZ22, LST, etc. His work lies in a region where we also find HGP codes, BB and GB codes, etc.

4 Material for peeling+guessing paper

Key points:

1. Adaptation of the Maxwell decoder to the quantum CSS case
2. Introduce the guessing mechanism
3. Introduce pruning and explain its role
4. Theoretical claims about performances: g ¿= d-s and g ¿= d-s+1
5. Complexity analysis of peeling + guessing O(N * 2**g)
    - discuss the O(n**2) post-processing that occurs only in benchmarking, not on operation
6. Applications (simulation results)
    - optimized HGP codes
    - BB codes
7. Non CSS generalization
8. Pauli error?
9. Comparison with existing methods
    - BPGD for erasures (Gokduman, Hanwen, Pfister)
    - Clustering (Pfister)
    - Degenerate erasure decoding (Kao-Yueh Kuo)
10. Rare events simulation? To go further down the p-¿0 regime

priority: 9 ¿ 6 ¿ 10
Paper outline:

4.1 Introduction

In quantum computing, quantum error correction techniques are essential to render quantum systems sufficiently robust to noise and capable of performing useful digital algorithms. Depending on the physical platform, different error models may more accurately describe the noise affecting the system. One particularly simplified and widely studied model is the quantum erasure channel, in which qubits are affected by errors at known locations; that is, each qubit can be erased with some probability independently, and the positions of all erased qubits are known to the decoder. An erased qubit can be modeled as a fully depolarized, maximally mixed state, and the underlying error affecting an erased qubit corresponds to a random Pauli operator determined upon syndrome measurement. Although simplified, the erasure channel provides valuable insights for analyzing the performance of quantum codes and decoding algorithms, and can be adapted to more realistic noise models. Moreover, some physical platforms can transform their native errors into erasures, which greatly simplifies the task of syndrome decoding.

In both the classical and quantum settings, the erasure decoding problem admits an optimal solution in terms of finding the most likely error configuration. This maximum-likelihood solution can be obtained by solving a linear system over the binary field, with Gaussian elimination as the most efficient general-purpose method. However, its cubic complexity in the number of qubits becomes prohibitive for large systems. This has motivated the development of iterative decoding algorithms with lower computational complexity at the expense of some loss in decoding performance. Such methods include message-passing algorithms, which exist in various forms for different noise models and have also been applied in the quantum setting. For the erasure channel, message passing reduces to the well-known peeling decoder.

In the classical case, the peeling decoder operates by progressively resolving erased bits, treating them as unknown variables in the parity-check equations. Whenever an equation contains a single unknown variable,

it can be solved directly, revealing the corresponding bit. As the algorithm progresses, more equations become solvable, and the procedure continues until either all erasures are resolved or no single-unknown equations remain, at which point the algorithm halts with a failure. In the quantum setting, for CSS codes, the peeling decoder is applied independently to the X and Z components of the code using the same initial erasure pattern; successful decoding requires that both parts be peeled successfully.

Numerous improvements to peeling have been proposed in the classical coding literature. One notable variant is the "Peeling + Guessing" decoder, which addresses situations where peeling stalls. In this approach, when no single-unknown equations remain, the decoder guesses one of the remaining erased bits and proceeds as in standard peeling. Each guess introduces branching in the decoding process, corresponding to the possible bit values. As decoding progresses, some branches may be discarded when inconsistencies arise (for example, when a fully determined equation evaluates incorrectly). This approach can be viewed as a form of list decoding, producing all candidate solutions consistent with the observed syndrome. If a single solution remains, the erasure is deemed correctable; otherwise, the decoder may return multiple equally likely solutions or declare a failure if the number of guesses exceeds a predefined limit.

Building on this line of work, analogous ideas can be extended to the quantum erasure channel for CSS codes. In such a scenario, after running a peeling-plus-guessing procedure, the resulting list of candidate corrections can be further reduced by computing their equivalence classes modulo the stabilizer group. If a single equivalence class remains, the decoding succeeds; if multiple equivalence classes appear, this indicates that the erasure covers a logical operator and is therefore not correctable. This additional step accounts for the degeneracy inherent to quantum stabilizer codes.

4.2 Algorithm description

Algorithm 1 Peeling + Guessing Decoder

Input: Subset E of erased qubits (binary vector), Syndrome vector, Guessing budget G

Initialize $g \leftarrow 0$
while $|E| > 0$ **do**
  Search for a dangling check
  if there is a dangling check **then**
    Call **DanglingCheckResolution** subroutine
  else
    if $g < G$ **then**
      Call **Guessing** subroutine
    else
      signal failure
    end if
  end if
end while
while there are empty equations **do**
  Enforce consistency of empty equations (cut inconsistent branches)
end while
for all remaining branches **do**
  Compute equivalence class modulo stabilizers
end for
if all branches belong to the same class **then**

  return any branch
else
  signal failure
end if

4.3 Theoretical results

1. At the asymptotic regime ($p \to 0$), the branching algorithm can match ML's performance up to a constant factor if the number of available guesses g is at least $d - s$, where d is the code distance and s is the stopping distance. Moreover, if $g \geq d - s + 1$, the performance of the branching algorithm will be asymptotically equivalent to that of ML.

Handwavy argument: For an erasure rate of p, the logical error rate is given by $\sum_{\mathcal{E} \subset [N]} p^{|\mathcal{E}|}(1-p)^{N-|\mathcal{E}|} \cdot [\mathcal{E} \text{ leads to a decoding failure}]$. Consider $\mathcal{E}$ such that $|\mathcal{E}| < d$. That implies that, for this instance of erasure error, the ML algorithm would succeed, i.e., $\mathcal{E}$ cannot contain a nontrivial logical operator. Under this assumption, the branching algorithm should only fail if it runs out of guesses before peeling $\mathcal{E}$ completely. Now, we know that for any erasure $\mathcal{E}'$ with $|\mathcal{E}'| < s$ (the stopping distance), the peeling algorithm is able to completely peel off $\mathcal{E}'$ and thus succeed. Since we assume that branching has at least $g = d - s$ guesses to spare, and it starts with an erasure pattern $\mathcal{E}$ of size $\mathcal{E} < d$, in the worst-case scenario peeling will block and trigger the branching mechanism as long as we have guesses left. However, after $d - s$ guesses, the size of the residual erasure will be less than $d - (d - s) = s$, and so it cannot still be a stopping set. That is, after at most $d - s$ guesses, the residual erasure gets too small to stop peeling from proceeding until the end. Thus, we get to the end with some number of branches (which may or may not have been cut along the execution), which we know must be all valid (that is, syndrome compliant and contained in the original erasure) and stabilizer-equivalent. To see why the remaining branches must be valid, just recall that branching has a consistency-check mechanism every time if finds a parity-check equation that had become dangling but whose dangling bit has been unerased before it was evaluated. Whenever these so-called empty equations are evaluated, they catch any branches that cause an inconsistency for it and trigger their cutting. Finally, to see why all the branches must be stabilizer-equivalent, just recall that, since $|\mathcal{E}| < d$, it cannot contain a nontrivial logical operator, and thus, all syndrome and erasure-compliant corrections must be stabilizer-equivalent. This way, we have that, for all $\mathcal{E}$ such that $|\mathcal{E}| < d$, $[\mathcal{E} \text{ leads to a decoding failure}] = 0$, and thus

$$p_L = \sum_{|\mathcal{E}| \geq d} p^{|\mathcal{E}|}(1-p)^{N-|\mathcal{E}|} \cdot [\mathcal{E} \text{ leads to a decoding failure}] = O(p^d).$$

Now, notice that the above argument gives us the same decay exponent d as for the ML algorithm, but we can't say the same about the constant, that is, how many erasure patterns $\mathcal{E}$ of size d will lead to a decoding failure for branching? Certainly at least as many as for ML, but could there be erasure patterns of size d for which ML succeeds and branching still fails even with $g = d - s$ guesses? Possibly yes. Nothing rules out the existence of d-sized erasure patterns that don't cover logical operators, but that contain s-sized stopping sets. To get rid of those, it suffices to add just one more guess, so $g \geq d - s + 1$ would be enough to match ML's asymptotic performance in both exponent and constant factor. It's good to highlight that it does not mean that the logical error rate for both algorithms will be exactly the same, because we can still have very large erasure patterns for which ML succeeds, but that block peeling sufficiently many times to exhaust all guesses lead to branching failure. However, those large patterns will have a negligible contribution to the logical error rate as $p \to 0$, in comparison to the others.

4.4 Simulation results

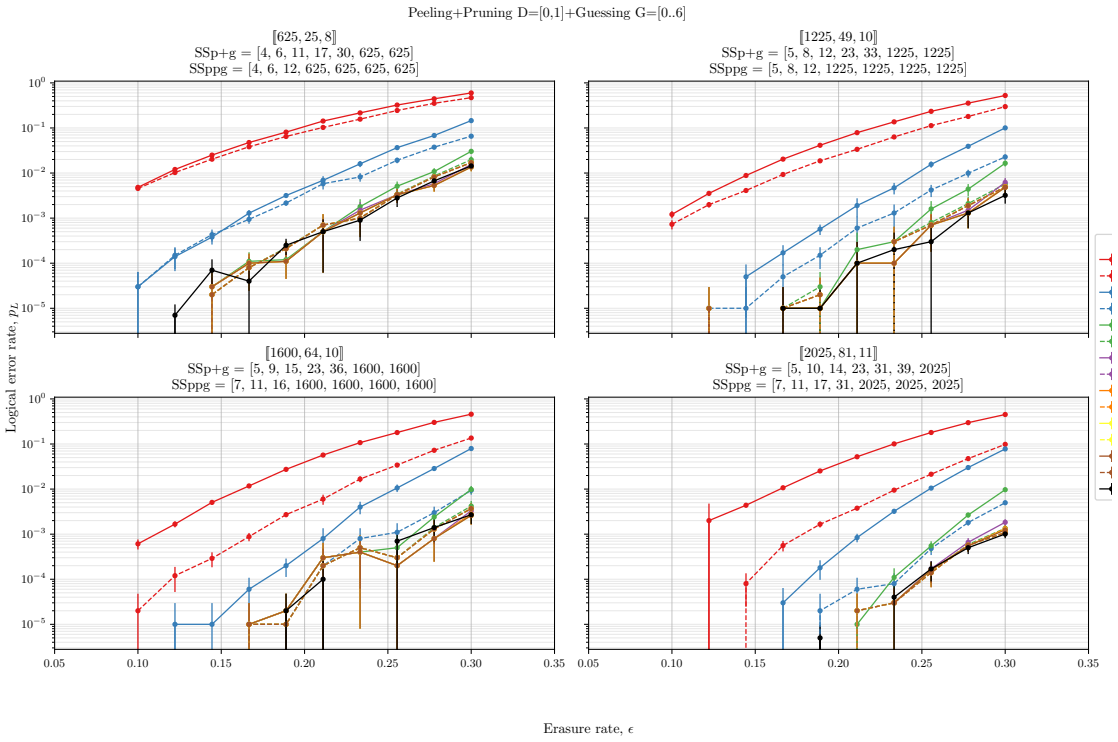


Figure 1: Performance of the PPG algorithm for some HGP codes

References

[1] Tom Richardson and Ruediger Urbanke. *Modern Coding Theory*. USA: Cambridge University Press, 2008. ISBN: 0521852293.

[2] M.G. Luby et al. "Efficient erasure correcting codes". In: *IEEE Transactions on Information Theory* 47.2 (2001), pp. 569–584. DOI: 10.1109/18.910575.

[3] Jean-Pierre Tillich and Gilles Zemor. "Quantum LDPC Codes With Positive Rate and Minimum Distance Proportional to the Square Root of the Blocklength". In: *IEEE Transactions on Information Theory* 60.2 (Feb. 2014), pp. 1193–1202. ISSN: 1557-9654. DOI: 10.1109/tit.2013.2292061. URL: http://dx.doi.org/10.1109/TIT.2013.2292061.

[4] Joschka Roffe et al. "Decoding across the quantum low-density parity-check code landscape". In: *Physical Review Research* 2.4 (Dec. 2020). ISSN: 2643-1564. DOI: 10.1103/physrevresearch.2.043423. URL: http://dx.doi.org/10.1103/PhysRevResearch.2.043423.

[5] Nicholas Connolly et al. "Fast erasure decoder for hypergraph product codes". In: *Quantum* 8 (Aug. 2024), p. 1450. ISSN: 2521-327X. DOI: 10.22331/q-2024-08-27-1450. URL: http://dx.doi.org/10.22331/q-2024-08-27-1450.

[6] Pavel Panteleev and Gleb Kalachev. *Asymptotically Good Quantum and Locally Testable Classical LDPC Codes*. 2022. arXiv: 2111.03654 [cs.IT]. URL: https://arxiv.org/abs/2111.03654.

[7] Anthony Leverrier and Gilles Zémor. *Quantum Tanner codes*. 2022. arXiv: 2202.13641 [quant-ph]. URL: https://arxiv.org/abs/2202.13641.

[8] Anthony Leverrier, Jean-Pierre Tillich, and Gilles Zemor. "Quantum Expander Codes". In: *2015 IEEE 56th Annual Symposium on Foundations of Computer Science*. IEEE, Oct. 2015, pp. 810–824. DOI: 10.1109/focs.2015.55. URL: http://dx.doi.org/10.1109/FOCS.2015.55.

[9] Antoine Grospellier et al. "Combining hard and soft decoders for hypergraph product codes". In: *Quantum* 5 (Apr. 2021), p. 432. ISSN: 2521-327X. DOI: 10.22331/q-2021-04-15-432. URL: http://dx.doi.org/10.22331/q-2021-04-15-432.

[10] Nadine Meister, Christopher A. Pattison, and John Preskill. *Efficient soft-output decoders for the surface code*. 2024. arXiv: 2405.07433 [quant-ph]. URL: https://arxiv.org/abs/2405.07433.

[11] Eric Sabo et al. *Weight Reduced Stabilizer Codes with Lower Overhead*. 2024. arXiv: 2402.05228 [quant-ph]. URL: https://arxiv.org/abs/2402.05228.

[12] Thomas R. Scruby, Timo Hillmann, and Joschka Roffe. *High-threshold, low-overhead and single-shot decodable fault-tolerant quantum memory*. 2024. arXiv: 2406.14445 [quant-ph]. URL: https://arxiv.org/abs/2406.14445.

[13] Stasiu Wolanski and Ben Barber. *Ambiguity Clustering: an accurate and efficient decoder for qLDPC codes*. 2024. arXiv: 2406.14527 [quant-ph]. URL: https://arxiv.org/abs/2406.14527.

[14] David Poulin and Yeojin Chung. *On the iterative decoding of sparse quantum codes*. 2008. arXiv: 0801.1241 [quant-ph]. URL: https://arxiv.org/abs/0801.1241.

[15] Guanyu Zhu. *A topological theory for qLDPC: non-Clifford gates and magic state fountain on homological product codes with constant rate and beyond the $N^{1/3}$ distance barrier*. 2025. arXiv: 2501.19375 [quant-ph]. URL: https://arxiv.org/abs/2501.19375.

[16] Elena Berardini et al. *Asymptotically good CSS-T codes exist*. 2024. arXiv: 2412.08586 [quant-ph]. URL: https://arxiv.org/abs/2412.08586.

[17] Alexander J. Malcolm et al. *Computing Efficiently in QLDPC Codes*. 2025. arXiv: 2502.07150 [quant-ph]. URL: https://arxiv.org/abs/2502.07150.

[18] R. Tanner. "A recursive approach to low complexity codes". In: *IEEE Trans. Inf. Theor.* 27.5 (Sept. 2006), pp. 533–547. ISSN: 0018-9448. DOI: 10.1109/TIT.1981.1056404. URL: https://doi.org/10.1109/TIT.1981.1056404.

[19] Peter W. Shor. "Scheme for reducing decoherence in quantum computer memory". In: *Phys. Rev. A* 52 (4 Oct. 1995), R2493–R2496. DOI: 10.1103/PhysRevA.52.R2493. URL: https://link.aps.org/doi/10.1103/PhysRevA.52.R2493.

[20] Andrew M. Steane. "Error Correcting Codes in Quantum Theory." In: *Physical review letters* 77 5 (1996), pp. 793–797. URL: https://api.semanticscholar.org/CorpusID:35977016.

[21] Daniel Gottesman. *Stabilizer Codes and Quantum Error Correction*. 1997. arXiv: quant-ph/9705052 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9705052.

[22] Eric Dennis et al. "Topological quantum memory". In: *Journal of Mathematical Physics* 43.9 (Sept. 2002), pp. 4452–4505. ISSN: 1089-7658. DOI: 10.1063/1.1499754. URL: http://dx.doi.org/10.1063/1.1499754.

[23] Austin G. Fowler et al. "Surface codes: Towards practical large-scale quantum computation". In: *Physical Review A* 86.3 (Sept. 2012). ISSN: 1094-1622. DOI: 10.1103/physreva.86.032324. URL: http://dx.doi.org/10.1103/PhysRevA.86.032324.

[24] Oscar Higgott and Craig Gidney. *Sparse Blossom: correcting a million errors per core second with minimum-weight matching*. 2023. arXiv: 2303.15933 [quant-ph]. URL: https://arxiv.org/abs/2303.15933.

[25] Adway Patra and Alexander Barg. *Targeted Clifford logical gates for hypergraph product codes*. 2024. arXiv: 2411.17050 [quant-ph]. URL: https://arxiv.org/abs/2411.17050.